

Investigating and Remediating Pathological Non-Executability of Public Computational Notebooks

Tien Nguyen · Waris Gill · Muhammad Ali Gulzar

Abstract Computational notebooks are the de facto platform for exploratory data science, offering an interactive programming environment where users can create, modify, and execute code cells in any sequence. However, this flexibility often introduces code quality issues, with prior studies showing that approximately 76% of public notebooks are non-executable. We argue that the traditional notion of executability—requiring a notebook to run *fully* and *without error*—is overly rigid, misclassifying many notebooks and overestimating their non-executability.

This paper investigates pathological executability issues in public notebooks under varying *notions* and *degrees* of executability. Notebooks, by construction, are incrementally and interactively executed, where each cell execution advances logic toward the notebook’s goal. Even partial improvements in executability can enhance code comprehension. With this insight, we first categorize notebooks into potentially restorable and pathological non-executable notebooks and then measure how removing misconfiguration and superficial execution issues in notebooks can improve their executability (i.e., additional cells executed without error). In a dataset of 39,717 public notebooks, containing 30,541 non-executable notebooks, only 36% are truly pathologically non-executable. For restorable notebooks, we use LLM-based input generation to fully restore 11.7% of previously non-executable notebooks. Among

This work was supported in part by Amazon - Virginia Tech Initiative in Efficient and Robust Machine Learning, 4-VA, and the National Science Foundation award 2106420. We also thank the Advanced Research Computing Center at Virginia Tech for their support in building and evaluating this work.

Tien Nguyen (Corresponding author)
Department of Computer Science, Virginia Tech, Blacksburg, VA, USA
E-mail: tiennguyen@vt.edu
ORCID: 0009-0005-4606-3976

Waris Gill
Department of Computer Science, Virginia Tech, Blacksburg, VA, USA
E-mail: waris@vt.edu
ORCID: 0009-0007-7100-4788

Muhammad Ali Gulzar
Department of Computer Science, Virginia Tech, Blacksburg, VA, USA
E-mail: gulzar@cs.vt.edu
ORCID: 0000-0002-8007-8662

the partially restored, we improve the executability of the notebooks by 43% and 30.9% by installing the correct modules and generating synthetic data. These findings challenge prior assumptions, suggesting that notebooks have higher executability than previously reported, with many offering valuable partial execution. Based on these insights, we design an automated test generation tool for executability, targeting the most common failure causes. We find that the generated tests accurately and preemptively reflect notebook executability by showing a low passing rate on non-executable notebooks (64.8%, on average) and a very high passing rate (100%) on fully executable ones.

Keywords Computational notebooks · Non-executability · Restoration · Mining software repositories · Testing

1 Introduction

Computational notebooks are the preferred tool for data analytics due to their portability and interactive capabilities. They enable easy sharing across various contexts and are often made publicly accessible, fostering reuse and adoption (Perez and Granger 2021; Quaranta et al. 2022a). In notebooks, developers write code and explanatory text in cells that can be run individually, displaying results instantly. An unexpected consequence of this interactive programming is non-reproducibility—more than 75% of the public notebooks are previously reported to be non-executable, let alone reproduce the promised output (Pimentel et al. 2019). Such a low level of executability is alarming as it diminishes the accessibility of public codebases and potentially causes code quality issues during reuse (Huang et al. 2022). Previous investigations have studied the executability and reproducibility issue of notebooks and reasoned about the presence of such issues (Head et al. 2019; Pimentel et al. 2019, 2021; Wang et al. 2020, 2021; Zhu et al. 2021). However, the findings of these studies have several limitations.

Problem. Previous studies categorize notebooks into (1) non-executable if they do not execute end-to-end in their current form or (2) executable if they execute successfully without any interference. There are two limitations to this categorization. First, a seemingly non-executable notebook may technically be executable but lack the appropriate execution environment (input file, packages, or execution order). Second, the binary notion of executability is overly rigid and stringent. Notebooks, by construction, are executed incrementally and interactively, similar to REPL (read-eval-print loop (Deutsch and Berkeley 1964)), where each cell execution logically progresses the notebook.

Even if a notebook is non-executable, its partial execution improves code comprehension during reuse. Furthermore, prior works report inconsistent findings on notebook non-executability (76% (Pimentel et al. 2019), 72.6% (Wang et al. 2021), 82.6% (Wang et al. 2020), and 47% (Zhu et al. 2021)) due to differences in the datasets analyzed. For instance, despite the large number of notebooks studied by prior work (Pimentel et al. 2019), most are unmanaged trial notebooks, often for personal, experimental projects rather than for reuse. Therefore, such findings do not generalize to popular notebooks (i.e., notebooks actively reused), demanding a fresh perspective on the non-executability of notebooks. We hypothesize that the count of executable notebooks is higher than previously reported, and even *pathologically*

non-executable notebooks are partially executable, offering valuable code reusability, code comprehension, and code repair opportunities. Finally, we notice that the computation notebooks ecosystem lacks standardized practices for software building and testing. This deficiency undermines the effectiveness and reproducibility of notebooks, even when developers intend to share and publish their work. Therefore, we need to design a method to help notebook developers perform a lightweight check to ensure that their notebooks are at least executable when shared.

Contributions. In this paper, we take a fresh perspective on notebook executability by investigating the reusability of public notebooks. We address the gaps in existing knowledge on notebook executability by making the following contributions. First, we introduce new notions and degrees of executability by separating misconfiguration from non-executability. To that end, we define *pathologically non-executable* notebooks that suffer from unresolvable fatal syntactic or runtime errors (such as bad indentation or attribute error), and *restorable* notebooks that are non-executable due to superficial misconfiguration errors (e.g., missing input, library, or execution order). Restorable notebooks are fully executable in a suitable environment (e.g., the original author’s local environment). Second, we introduce fine-grained metrics of executability that quantify the reuse potential of non-executable notebooks. For non-executable notebooks that are otherwise deemed unusable, we measure their *partial executability* based on the proportion of successfully executed cells, an aspect that has never been studied. Third, we conduct this investigation on actively reused and adopted notebooks. We follow prior work that uses GitHub stars as a proxy for community visibility Borges and Valente (2018) to rank notebooks during data collection. Doing so mitigates any bias from unmanaged, rarely used public notebooks. Lastly, we demonstrate the effectiveness of LLMs in addressing misconfiguration issues (e.g., by generating synthetic input data) in restorable notebooks.

We realize the aforementioned contributions by building an automated measurement framework for notebooks. Each notebook is evaluated in a sandboxed environment with all required execution environments as stated in the parent repository. We leverage a combination of static and dynamic error checking to extract the causes of non-executability in notebooks. Static analysis on notebook code cells identifies errors such as structural, syntax, and variable-undefined errors. Dynamic analysis identifies runtime errors such as missing libraries, packages, and type errors. This stage returns a categorized list of pathologically non-executable notebooks and restorable notebooks, and their respective list of errors. LLMs are improving code generation and understanding (Lin et al. 2024; Nejjar et al. 2024). To investigate the reusability of restorable notebooks, we utilize open-source LLM Gemma3 (Google 2025) for error-driven notebook restoration. For example, error reports indicating missing modules or input files are combined with code cells to prompt the LLM to identify the correct module names for installation or generate synthetic input data, respectively. We iteratively restore and error-check the notebook until it is fully executable or an unfixable error is found.

To address the non-executability issue at its source, we leverage the takeaways from this empirical investigation and design an automated executability test generation tool that synthesizes unit tests verifying that the notebooks under development remain reusable. During development, we statically and dynamically monitor notebooks’ development by instrumenting JupyterLab and identifying properties that correspond to the most common reasons for the non-executability of notebooks. For instance, when the input file is read in the notebook’s code cell, it prompts auto-

mated unit test generation, which verifies that the file is available and accessible within the context of the code cell when shared. Similarly, we generate unit tests for all imported modules used in the notebook and attempt to retrieve their versions. When a required dependency is missing, we recommend (as a failing test) that authors include it in a `requirements.txt` file to guarantee that others can reproduce the execution environment. By guiding authors to detect and correct executability issues before publishing, our test generation process promotes the long-term reliability and usability of public notebooks.

Study Results. We collect a dataset of the most popular 39,717 public notebooks from a stratified sample of 3,375 GitHub repositories with four or more stars in 2024. Using non-executability criteria from prior work, we found that over 76.9% of those notebooks are non-executable if executed as is, which is consistent with the findings of prior studies (Pimentel et al. 2021; Wang et al. 2021). We investigate the top two reasons for non-executability: lack of appropriate input files and missing modules, packages, or libraries. Such notebooks are potentially restorable. Surprisingly, we find that **only 36% of all notebooks in the dataset are pathologically non-executable** due to unrestorable executability issues, and the rest are either executable or potentially executable given a suitable execution environment, i.e., 53.2% of non-executable notebooks can potentially be restored. This validates our hypothesis that pathologically non-executable notebooks are significantly lower than previously reported in prior work. Possible reasons include a skewed notebooks dataset, a misconfigured experiment environment, and the strict notion of executability. Interestingly, our error-driven, LLM-guided fix successfully restores the full and partial executability of 37% of the notebooks for which it found the necessary module and synthetic input data. It partially improves the executability of restorable notebooks by 26.8% cells per notebook. Prior studies (Pimentel et al. 2019, 2021) have considered these notebooks completely non-executable. Even pathologically non-executable notebooks are not entirely non-executable. They can be successfully executed up to 31.3% of cells, on average. These findings show that traditional notions of executability underestimate notebooks’ reusability, mainly due to misconfigured notebooks, some of which can be fixed automatically, and that most notebooks can still offer numerous code reuse and restoration opportunities.

To assess the effectiveness of auto-generated tests in exposing executability issues, we collect a dataset of fresh notebooks with at least one commit from 2025. We follow the same procedure as with the previously collected dataset. This new dataset comprises a stratified sample of 3,169 notebooks from 192 repositories. For all executable (including notebooks restored using our LLM-driven approach) and non-executable notebooks, we measure the accuracy of auto-generated tests to determine whether the tests can successfully capture executability issues when present and whether they pass when there are no executability issues. We find that 34.5% of notebooks fail environment dependency tests, 9.2% fail input file dependency tests, and 69% fail linear execution tests. After applying our error-driven, LLM-guided fix, we reduce the test failure rate to 22.5%, 3.8%, and 64.8%, respectively, demonstrating the adequacy of our tests in detecting executability issues. These findings confirm that the majority of executability failures stem from missing dependencies rather than pathological errors. By applying our automated test generation tool, notebook authors can proactively evaluate and correct executability issues before publishing, thereby improving the long-term reusability and executability of public notebooks.

We summarize the contributions of this work below.

- We take a fresh look at the reusability of public notebooks by exploring varying interpretations of executability. Our findings offer insights into the partial executability of seemingly non-executable notebooks and distinguish superficial misconfiguration errors from pathological errors.
- This is the first work to incorporate notebook popularity when assessing executability, contextualizing findings to frequently reused and adopted notebooks.
- We report the feasibility of lightweight restoration strategies in improving notebooks’ executability by addressing three of the top executability issues. We also demonstrate that LLMs can be effective in generating synthetic sample input to advance the partial executability of notebooks.
- We package our findings and study framework in an automated test generation tool that notebook developers can use to assess the executability of their notebooks before publishing them.
- Executability is the first step toward reproducibility. A significant portion of cells in pathologically non-executable notebooks remains executable, unlocking the potential for advancing code reuse and dynamic analysis. Partial executability can serve as a valuable criterion to assess a notebook’s restoration potential.

EMSE Extension Contributions. This work extends our previous study (Nguyen et al. 2025). The major extensions presented in this paper are as follows:

- We designed and developed an automated executability test generation tool, informed by findings from our earlier empirical studies (Section 6).
- We applied this tool to a newly collected dataset and evaluated notebook executability based on the generated test cases (Section 6.1).
- We made the source code for the test generation tool publicly available on GitHub.
- We analyzed the top 10 most common errors in notebooks across different star ranges, observing a decrease in occurrence as star count increases (Section 5.1.3).
- We performed a longitudinal analysis between the 2024 and 2025 datasets, finding that the newer dataset has a higher proportion of executable notebooks.
- We expanded our literature review to include recent work on notebook testing tools (Section 9.4).
- We updated our empirical study using the same dataset as before, reflecting its latest state; some repositories have become unavailable, while others have resolved previous issues, resulting in updated findings.

Corresponding revision to incorporate reviewer’s suggestion. We update our empirical study using the same dataset and incorporate a more recent LLM model, Gemma3 (Google 2025). While the dataset has evolved over time (e.g., repository availability and fixes), leading to changes in reported numbers, the overall patterns and insights remain consistent. Using an up-to-date model ensures that our findings reflect current LLM capabilities without materially altering the observed trends.

2 Motivation

This section presents two case studies demonstrating that seemingly non-executable notebooks can potentially be restored with minor reconfiguration, and pathologically non-executable notebooks still offer valuable partially executable code.

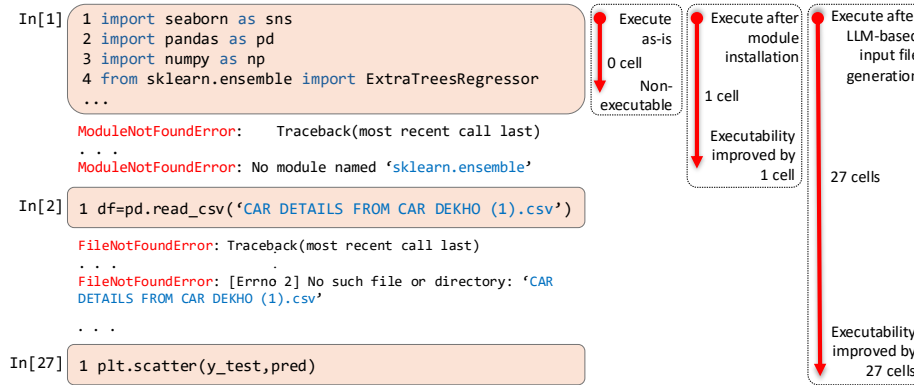


Fig. 1 Notebook (Choudhari 2021) is initially non-executable due to a missing module. Its executability improves by 1 cell by resolving the required dependency. Its executability can be fully restored by 27 cells by generating a missing input file for the notebook

2.1 Case Study 1: Restoring Complete Executability

The notebook “car price prediction.ipynb” (Choudhari 2021) implements a prediction task on different machine learning regression models. This notebook is hosted by the author Pratik Choudhari’s GitHub open-source repository “AlgoCode” with 130 stars. It consists of 27 code cells. When this notebook is executed as is, it results in a “ModuleNotFound” error in cell 1, attempting to import the module `sklearn`. The original execution counts in the notebook suggest that the notebook was executable in the author’s environment before publishing it to GitHub. However, this requirement is neither declared in a REQUIREMENTS file nor is the file included in the repository. Prior studies would classify this notebook as non-executable (Pimentel et al. 2019, 2021). This notebook aims to demonstrate the logical steps required to create different models for regression tasks. Our observation is that, for such a notebook, improving executability can help serve the notebook’s purpose, which can be done by providing the required environment dependency. To do so, we try to install the module `sklearn` into the environment with the correct name `scikit-learn` with the help of the LLM engine Gemma3 (Google 2025). The notebook’s executability improves by 1 cell until it encounters “FileNotFound” error when it attempts to read the input file “CAR DETAILS FROM CAR DEKHO (1).csv” in cell 2. Again, this file was accessible during the author’s execution, as suggested by fully linear execution counts. However, the input file is not provided along with the repository. This would also be considered non-executable according to previous works. This classification is analogous to considering “javac” non-executable if no input `.java` file is provided. We could improve the executability of this notebook by generating a synthetic input file. For this case, we package the notebook’s code, Python error description, and the notebook’s documentation into a prompt and query Gemma3 to generate an input file with a correct relative path to the notebook with synthetic content. This synthetically generated input file results in full executability of all 27 cells, i.e., improving the executability of the previously non-executable notebook by

100% as illustrated in Figure 1. This is a prime example of how non-executability is often over-classified when the input file or requirement file is not provided, but may be available locally to the original author.

2.2 Case Study 2: Improving Partial Executability

The notebook “DinosaurIsland--Characterlevellanguagemodelfinal-v3.ipynb” from the GitHub repository “deep-learning-coursera” (Atienza 2018) is demonstrating a deep learning tutorial. It has 129 stars and a total of 14 code cells. When executed in linear order, the notebook first encounters a “ModuleNotFound” error in cell 1 due to the absence of module `utils`. We resolve this error by installing the required module. A follow-up execution raises a “NameError” in cell 6 due to the undefined function `softmax()`. After reviewing the notebook, we noticed that the markdown cell claimed that the function was provided but was missing. The original notebook shows a valid output, which means that the author successfully executed it. This is a common case where program states are shared between different executions, even if the corresponding code is moved or deleted. Even if the notebook was executed end-to-end before pushing to GitHub, the error would not have surfaced, as the function definition is still present in the current Python kernel. However, such program states are not shared with the notebook itself, resulting in a “NameError” in a new Python kernel.

Our analysis indicates that defining the undefined name with an appropriate implementation could significantly improve the notebook’s executability. Therefore, we prompt LLM with the error report and the code cells to generate a definition for this function and insert a new code cell containing the definition right before its use. This LLM-generated code cell requires a module import for `tensorflow`. Again, we were able to install it in the environment. These simple installations of modules and additions improve the executability of the original notebook by 7 cells until it encounters “AttributeError” in cell 8. This is a tutorial notebook, and the first half illustrates valuable deep-learning knowledge in sequence models. Thus, partial executability offers added value.

3 Research Questions

According to the traditional definition of executability, a notebook is considered **executable** if it does not trigger any error throughout its complete top-down execution. A **non-executable** notebook, on the other hand, fails to execute due to an error or exception. In this paper, we relax this notion of executability by dividing it further into **pathologically non-executable**, **non-executable but restorable**, and **executable**. When a notebook’s executability is hindered by unresolvable errors, e.g., syntax or indentation errors, it is **pathologically non-executable**. If a notebook is executable in the original author’s execution environment but fails to execute in another environment due to issues such as missing input files, execution orders, or execution environments, it is called a **non-executable but restorable** or **misconfigured** notebook, which is not executable but can be fully or partially restored.

Similarly to different notions of executability, we also introduce different degrees of executability. Instead of considering it as a binary metric, we measure executability on a continuous spectrum, defined as a ratio between the number of cells executed until the first error and the total number of cells in a notebook. 100% executability refers to fully executable notebook, whereas $<100\%$ refers to partially executable notebooks. A fully executable notebook does not guarantee reproducibility, i.e., a notebook must produce the same outputs as the original notebook. Reproducibility is beyond the scope of this work and is often not required for data-centric notebooks, as they are expected to produce different outputs on different data. We explore the following research questions:

- **RQ1:** What are the common causes of non-executability in notebooks?
- **RQ2:** How many non-executable notebooks are pathologically non-executable, and how many can be restored?
- **RQ3:** To what extent can pathologically non-executable notebooks be executed?
- **RQ4:** Can LLM-based restoration strategies enhance notebook executability?
- **RQ5:** How can we assist developers in automatically ensuring full executability of computational notebooks during development?

4 Empirical Analysis Methodology

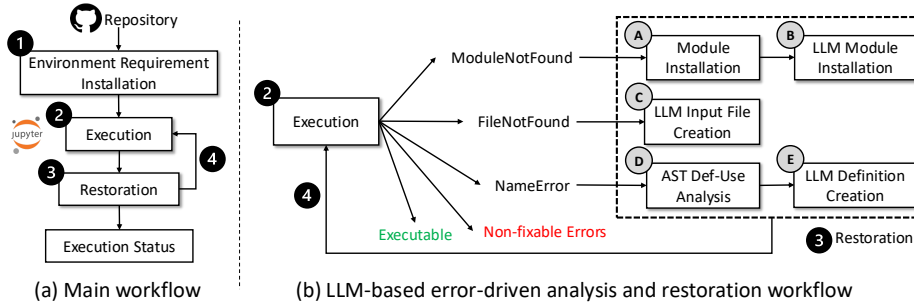


Fig. 2 LLM-based workflow for error-driven notebook executability analysis and restoration

Figure 2 (a) illustrates the intermediate stages of our analysis process. Starting with a repository, we retrieve all notebooks and perform a lightweight static error check to identify issues that may impede notebook execution. Next, we automatically analyze the repository to extract any required environment file provided, enabling us to configure the appropriate environment for notebook execution. During execution, we log the first recorded error and incrementally apply targeted restoration strategies based on the error type, measuring improvements in executability at each stage.

4.1 Analysis Approach

4.1.1 Initial Error Checking

For a notebook to be executable, it first needs to be compilable. As a first step, we look for compilation errors by applying lightweight error checking to all notebooks within each repository, uncovering any compilation issues, such as syntax or indentation errors. This preliminary filtering step assesses executability without running the code, allowing us to exclude notebooks that require extensive code-level modifications for restoration and are thus deemed pathologically non-executable. To access the content of the notebooks, we use the `nbformat` module. While performing static analysis of the notebook’s code, we detect and omit notebooks from further investigation that (1) cannot be read due to corruption and encoding problems; (2) contain no code cells (with or without markdown cells), or all code cells do not hold any content; and (3) written in programming languages other than Python 3 (such as Python 2, Julia, R, or C).

4.1.2 Dynamic Error Checking

In step ② of Figure 2 (b), we perform dynamic error checking to detect and categorize potential runtime errors. We use Papermill (nteract contributors 2018), which allows us to execute the notebook and detect and report any errors encountered during execution. We use Python 3 as the execution kernel and document the first error that halts the execution. We categorize the execution status of each notebook into the following groups:

1. **Executable:** The notebook is fully executable without encountering errors.
2. **FileNotFound:** The notebook writes to or reads from a file or directory that is unavailable during execution.
3. **ModuleNotFound:** The notebook imports unavailable modules, packages, or libraries.
4. **NameError:** The notebook uses a variable, function, or class without defining or importing it.
5. **Others/Non-Fixable:** The notebook encounters errors not covered by the above categories. We also identify and categorize these errors accordingly.

The categories reflect a natural progression of notebook adoption and fixes, as shown in Figure 2. If a notebook is found to be fully executable after this execution process, we mark it as executable. If it is non-executable due to missing input files, required modules, or undefined names, we iteratively apply restoration strategies (Figure 2 (b)) and perform dynamic error checking to remove as many executability issues as possible.

For notebooks with “NameError,” we perform a static *def-use* analysis to localize the root cause behind undefined names. Like traditional *def-use* analysis, we build a specialized AST visitor that tracks variable definition and usage with precise scope awareness within each cell. Figure 3 shows how *def* and *use* sets look for two notebook cells. Note that while maintaining the definition set, we consider variable binding, access, and scoping rules for Python to avoid any false positives and false negatives during the analysis. Our definition set extends beyond variables to include imports, functions, classes, and control flow elements.

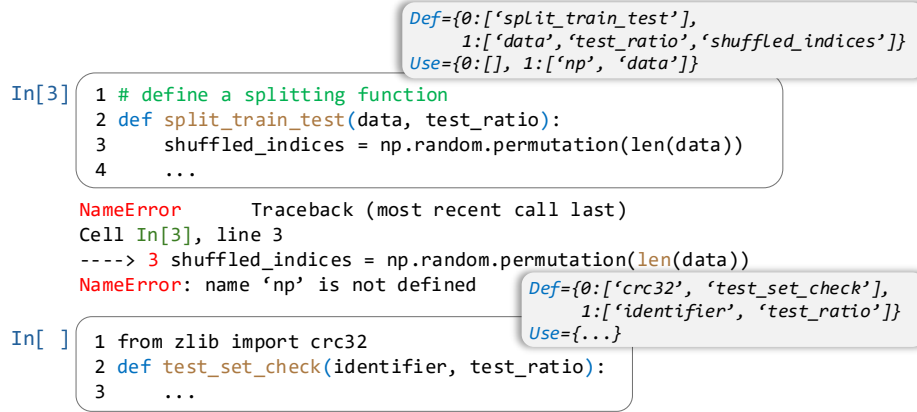


Fig. 3 Def-use lists for the first two cells in notebook (Hosni 2023)

Next, we use *def-use* sets to isolate the location of undefined names in the notebook and search nearby cells to locate the definition. Finally, the framework returns any detected definition of the undefined variable and its cell location. If the variable’s definition appears later in the notebook than in its initial use, we categorize the notebook as *defined after use* and return the cell number where the definition is found. If no definition exists throughout the notebook, we classify it as *undefined*.

4.2 LLM-Based Error-Driven Restoration

This section explains the lightweight use of LLMs to synthesize execution environments, enabling improved measurements of notebook restorability. This addresses undefined name issues from incorrect execution orders or the absence of definitions, generates synthetic yet syntactically valid input, and installs external modules. To improve precision and reduce ambiguity, we guide LLM output using structured schemas. Instead of free-form text, the LLM is instructed to return well-formed JSON objects. This approach improves automation reliability and reduces errors caused by misinterpreting responses.

4.2.1 ModuleNotFound Error

Most of the repositories do not provide information on environment requirements, and if they do, those modules are often outdated (Wang et al. 2021; Zhu et al. 2021). Notebooks with “ModuleNotFound” errors are restorable since they were originally executable but failed to execute in a new environment. We attempt to reestablish the execution environment for the notebooks by inspecting the REQUIREMENTS/INSTALL instructions and installing the required dependencies in the execution environment before executing any notebooks. During execution, if a notebook returns a “ModuleNotFound” error, we extract the missing module name from the error message and construct a terminal command `pip install <missing module>` (A of Figure 2). If the installation fails due to an incorrect or deprecated module name,

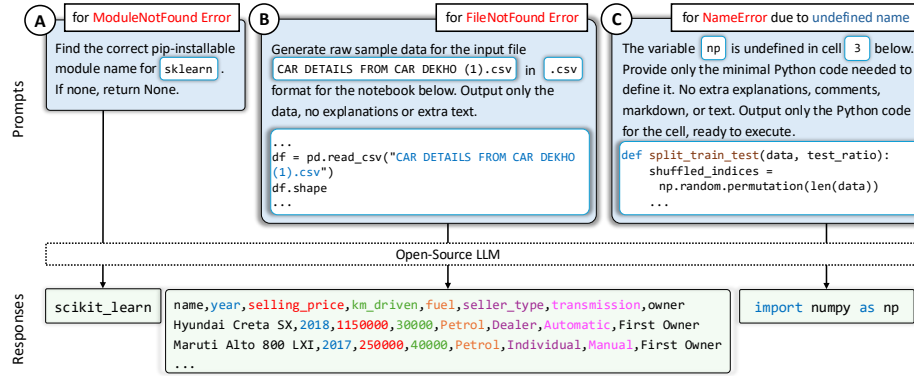


Fig. 4 Example LLM prompts and responses for different error types

we use LLM to obtain the correct and updated name for the corresponding missing module (④-B of Figure 2 and LLM prompt in Figure 4-A) and retry the installation.

4.2.2 FileNotFoundError

“FileNotFound” errors often occur when notebooks attempt to access and read unavailable input files or directories. We use LLM to generate synthetic input data tailored to the notebook’s semantics, attempting to resolve “FileNotFound” errors while avoiding additional runtime issues (④-C of Figure 2 (b)). Our insight is that the code cells in notebooks contain sufficient information for LLMs to generate syntactically correct input data needed for execution. Since our goal is executability rather than reproducibility, syntactically correct synthetic data are sufficient to address the “FileNotFound” error and restore notebook executability.

Concise contexts improve LLM performance (Ramlochan 2024). Thus, we provide contexts of the notebook error, such as missing file path and type, which guide LLM on file syntax, correct data type, and content. Instead of the `.ipynb` file, we provide the source code of all cells in the prompt. This conversion from `.ipynb` to Python also removes noise that may misguide or overwhelm the LLMs. Figure 4-B shows this prompt and the response of the LLM for the example mentioned in 2.1.

4.2.3 NameError

For a non-executable notebook due to undefined names, we extract the name and the specific cell in which the error occurs, and the cell where the name definition is located, or indicate if it cannot be found (④-D of Figure 2 (b)). We then prompt LLM to address this issue as illustrated in ④-E of Figure 2 (b). The prompt also includes the code cells and the error type (see Figure 4-C). The LLM’s response is used to rewrite the notebook, which is analyzed again for additional errors. However, the changes introduced by LLMs may change the notebook’s semantics. Since test cases are almost nonexistent in notebooks (only 1.54% of notebooks have tests (Pimentel et al. 2021)), it is challenging to verify semantic accuracy. Even if test cases are available, we must first make the notebook executable to enable testing and detect any semantic inconsistencies introduced by LLM-based modifications. Our position

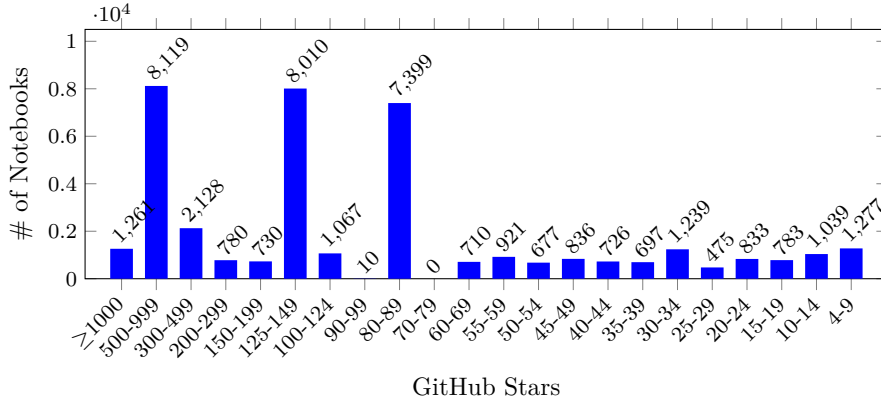


Fig. 5 Distribution of notebooks and their GitHub stars

is that addressing executability is a prerequisite for reproducibility, which requires cell executability even if it causes semantic changes.

5 Empirical Results

The following sections present the empirical findings of the notebook executability analysis.

Dataset. We focus on two criteria to collect notebooks to investigate our research questions. First, we aim to sample notebooks that have garnered significant community visibility and adoption. Second, notebooks should be the most recent version at the time of the dataset collection, as the computational notebook ecosystem has evolved in recent years with the advent of AI and new tooling.

To operationalize adoption, we use GitHub stars as a proxy, consistent with prior work showing that star counts influence developers’ decisions when selecting software for reuse (Borges and Valente 2018). Stars provide a simple and widely recognized signal of community interest and can be viewed as a conservative lower bound on adoption, since many users may reuse repositories without explicitly starring them. In contrast, alternative signals such as commit frequency, issue/PR activity, and number of contributors primarily capture development and maintenance effort rather than actual usage, while metrics such as CI activity or repository traffic may overemphasize repositories under active development or include non-notebook artifacts, introducing noise for our purpose.

We use the GitHub API to search for Jupyter notebooks with parameters including language and star range. We exclude irrelevant files that are either unreadable, written in non-Python languages, authored in Python 2 (deprecated since 2020), or automatically generated checkpoint files (`.ipynb_checkpoints`). In addition, we exclude notebooks that fail Python AST parsing due to syntax errors. A potential concern is that such notebooks might be recoverable using automated repair tools and, therefore, should not necessarily be excluded. To evaluate this, we applied a suite of widely used Python linters and formatters (e.g., `black`, `ruff`, `autopep8`, `yapf`, `flake8`, `pylint`, `mypy`, and `prospector`) to all syntax-invalid notebooks. However,

only 756 of 27,846 initially invalid notebooks were successfully repaired to syntactic validity, while the vast majority, 27,090, remained invalid. This is because such tools operate on valid ASTs and cannot fix parse-time errors (e.g., `IndentationError`). Given their limited impact, we consider our AST-based filtering sufficient and exclude these notebooks from the dataset.

We then perform stratified sampling at the *repository level*, selecting up to 1000 repositories per GitHub star “tier” (e.g., ≥ 1000 , 500–999, 300–499, and so on), with logarithmic tier sizes to account for the long-tailed distribution of repository popularity. Because repositories vary substantially in the number of notebooks they contain, this strategy results in unequal numbers of notebooks per tier. In particular, highly starred repositories tend to contain more notebooks, leading to higher notebook counts in those tiers. Additionally, star-range queries were processed sequentially under time constraints, which introduces minor variation in collection completeness across tiers.

This process yields approximately 318,000 notebooks. Due to prohibitive compute costs of LLMs, we take a 12% sample of total notebooks, resulting in 39,717 notebooks from 3,375 repositories. Figure 5 reflects these effects, where variations in notebook counts across tiers appear as spikes.

Analysis Environment. We establish a separate virtual environment for each repository. Sandboxing the execution environments eliminates the risk of mismatched versions of Python modules, side effects from executing the notebooks, and conflicts stemming from shared dependencies. We implement sandboxing using isolated Conda environments, where each repository is executed in a clean environment independent of all others, ensuring dependency isolation and preventing cross-repository interference. While more heavyweight containerization approaches, such as Docker, could further strengthen isolation, our Conda-based setup provides sufficient isolation for reproducible execution at scale. We use Python version 3 as the execution kernel. Before analyzing a notebook, we locate all possible requirement files (if available) in the repository that list the necessary modules or packages and install them in the execution environment (in ❶ of Figure 2). Note that the installation of required modules or libraries may fail due to multiple reasons, such as malformed files, incompatible versions, or external tool requirements (Pimentel et al. 2019). For the feasibility and scalability of this study, we also set an execution timeout of 5 minutes. We use a state-of-the-art open-source LLM called Gemma3 (Google 2025).

5.1 RQ1: Causes of Non-Executability

Our first goal is to identify the non-executable notebooks and the reason behind their non-executability. We first set up the execution environment by incorporating the installation instructions in the parent repository. This attempt at executing the notebooks results in 9,176 (23.1%) executable notebooks, and 30,541 (76.9%) non-executable notebooks. Our findings are consistent with previously reported 76% by Pimentel et al. (2019). We identify two factors that continue to shape executability outcomes. First, repositories often lack complete and detailed requirements files. Even when requirements are present, we still encounter environment misconfiguration errors in nearly 31.3% of the notebooks. Second, prior studies included a large proportion of obsolete, simple, and unmanaged notebooks. Such notebooks, typically created for quick experimentation, are short and less dependent on external libraries,

Table 1 Top 10 Common Errors in Notebooks

Error Type	# of Notebooks	% w.r.t Non-Executable	% w.r.t Dataset
ModuleNotFoundError	12,430	40.7%	31.3%
ImportError	5,281	17.3%	13.3%
FileNotFoundError	3,957	13%	10%
ValueError	3,556	11.6%	9%
TypeError	2,750	9%	6.9%
AttributeError	2,392	7.8%	6%
NameError	1,269	4.2%	3.2%
KeyError	703	2.3%	1.8%
StdinNotImplementedError	536	1.8%	1.3%
IndexError	468	1.5%	1.2%

which lowers the likelihood of “FileNotFound” and “ModuleNotFound” errors but results in higher rates of “NameError”, as observed in (Pimentel et al. 2019). We hypothesize that once environment configuration issues are addressed, the executability of more complex notebooks will improve, while low-quality notebooks may remain less executable.

Next, we investigate and categorize why 76.9% notebooks cannot be executed and find the main reasons that hinder their execution. Table 1 presents the top 10 issues encountered in the notebooks. Among them, “ModuleNotFound” and “FileNotFound” errors are the most frequent, making up 31.3% and 10% of the entire dataset, respectively. Other common exceptions include “ValueError” (9%), “TypeError” (6.9%), “AttributeError” (6%), and “StdinNotImplementedError” (1.3%).

5.1.1 Execution Environment

In our analysis, 12,430 notebooks encountered “ModuleNotFound” errors during execution when executed in a fresh environment. This accounts for 40.7% of the total non-executable notebooks. These errors arise when the required modules or packages are not available in the package manager (e.g., `pip`) or cannot be located within the execution environment in the case of custom packages and libraries. Such mismatches highlight the importance of ensuring compatibility and consistency across execution environments to facilitate seamless notebook execution across different computing setups. Prior work (Pimentel et al. 2019) reports that 20.7% of notebooks failed due to “ImportError,” whereas our analysis finds that only 13.3% of notebooks in our dataset encounter this exception. ImportError can occur for various reasons, such as issues within the module or missing dependencies, while the “ModuleNotFound” error specifically occurs when Python cannot locate the module at all. Introduced in Python 3.6, the “ModuleNotFound” error clarifies this distinction, making it easier for developers to handle missing modules.

5.1.2 External Input Data

We find 13% of the total non-executable notebooks affected by the “FileNotFound” error. Additionally, when the notebook requires user-typed input, notebook execution will raise a “StdinNotImplementedError.” We do not attempt to restore the

executability of those notebooks, as they necessitate dynamic interactions, which are not feasible in an automated execution environment. Overall, we detect a total of 1.8% initially non-executable notebooks (1.3% of the entire dataset) that required user input during their execution. This finding is consistent with the report by Pimentel et al. (2019), which indicates approximately 1.1% of notebooks are affected by this type of error.

5.1.3 Missing/Disordered Variable Definitions

Table 2 Percentage of notebooks with different errors in different star ranges

	Star Range				
	≥ 1000	500-999	100-499	10-99	4-9
ModuleNotFoundError	20.5%	33.6%	30.9%	30.9%	36.6%
ImportError	12.4%	13.3%	13.7%	13.5%	6.7%
FileNotFoundError	1.4%	8.6%	9.8%	11.5%	8.9%
ValueError	12.2%	7.1%	8.5%	10.0%	8.5%
TypeError	10.9%	6.3%	6.9%	7.0%	6.3%
AttributeError	8.2%	5.4%	6.0%	6.3%	5.3%
NameError	2.0%	2.7%	3.1%	3.3%	7.0%
KeyError	1.0%	2.0%	1.8%	1.7%	2.6%
StdinNotImplementedError	0.4%	1.6%	1.2%	1.3%	3.4%
IndexError	0.7%	0.8%	1.5%	1.2%	0.9%

Table 2 shows the distribution of the most common errors (see Table 1) across different repository popularity ranges. In general, we observe that as repository popularity increases, the frequency of most errors decreases, indicating that highly starred notebooks tend to have fewer issues and are more executable. For example, “ModuleNotFoundError” occurs in 36.6% of the lowest-star notebooks (4–9 stars) but decreases to 20.5% for notebooks with 1000 or more stars. Other errors, such as “FileNotFoundError” or “KeyError”, follow a similar trend, decreasing for the highest popularity notebooks. Some exceptions, like “ValueError” and “TypeError”, slightly increase for the highest star range.

“NameError” in a notebook can arise due to the use of a variable, library, function, or class name that was not previously defined. Among the 30,541 notebooks that are non-executable, we identify 1,269 notebooks containing at least one instance of this error, accounting for 4.2% (3.2% of total), as shown in Table 1. While “NameError” affects only a small portion of our dataset, prior work ranks it among the top two errors, reporting it in 14.53% of notebooks (Pimentel et al. 2019). Further investigation of Table 2 finds that “NameError” is less prevalent in more popular notebooks, with its occurrence decreasing as the star increases. Thus, “NameError” is predominantly found in low-popular notebooks, which are less actively reused and frequently maintained, allowing such errors to persist. This provides concrete evidence that “NameError” tends to exist in less frequently used notebooks. Therefore, a higher “NameError” rate reported by Pimentel et al. (2019) suggests that *the dataset of notebooks studied in earlier work includes a significant number of low-popularity and rarely managed and reused notebooks*. For example, the notebook (JadFatTail

2020), with only four stars, resembles a trial notebook that defines numerous functions without any documentation, ultimately leading to an undefined function error.

5.1.4 Experimentation Notebooks

Notebooks are often used as programming playgrounds, and users who are new to programming often use them to dabble with programming. Notebooks resulting from such exercises are not managed and not expected to be shared and, thus, by definition, are incomplete. Similar to these notebooks, we find that 2.4% of notebooks in our dataset contains an infinite while loop to demonstrate concepts such as infinite loops, large-scale training campaigns, or I/O operations over the network. Executing such notebooks may result in “TimeoutError” issues. Notebooks with “TimeoutErrors” are not analyzable; thus, we exclude those from Table 1.

We further investigate the original execution patterns of notebooks to understand how they were run before being pushed to GitHub. These execution patterns provide a deeper insight into why many notebooks are not fully executable. Information on prior cell execution order is often stored in the `.ipynb` format under the JSON field `execution_count` for each cell. In the cases where such order was available, we observe several recurring execution count patterns: (1) *linear execution*, where all code cells are run in strictly increasing order; (2) *repeated execution*, where one or more cells are executed multiple times; (3) *skipped execution*, where some cells are never run; (4) *non-linear execution*, where cells are executed out of order; and (5) *no execution*, where no code cells are executed at all.

Our empirical study shows that many notebooks exhibit more than one execution pattern—for example, simultaneously containing both skipped and repeated cells. Such cases often arise when notebooks are used for testing, exploration, or non-sequential development. Among all notebooks in our dataset, 27.2% display mixed execution patterns. The most common individual pattern is *skipped* (34.3%), followed by *repeated* (31.1%) and *no execution* (24.9%). These findings suggest that most public notebooks are not executed in a fully linear manner before being published to GitHub. Together with the empirical results above, this further highlights how authoring practices—such as inconsistent execution and incomplete dependency management—contribute to non-executability in public notebooks.

Takeaway RQ1. Our analysis shows that the primary causes of non-executable notebooks are missing dependencies and external data. High-quality notebooks have fewer fundamental errors like “NameError”, indicating better code maintenance and frequent usage.

5.2 RQ2: Pathological Non-Executability

Resolving errors like compilation errors, syntax errors, indentation errors, and runtime errors like “OSError” and “HTTPError” require significant code refactoring to align with the intended semantics. Without the knowledge of such semantics, the executability of such notebooks cannot be correctly restored. Notebooks requiring extensive intervention to restore executability are categorized as *pathologically non-executable*. Notebooks with errors that can be resolved by configuring the execution environment are classified as non-executable but *restorable*. For instance, as in Case

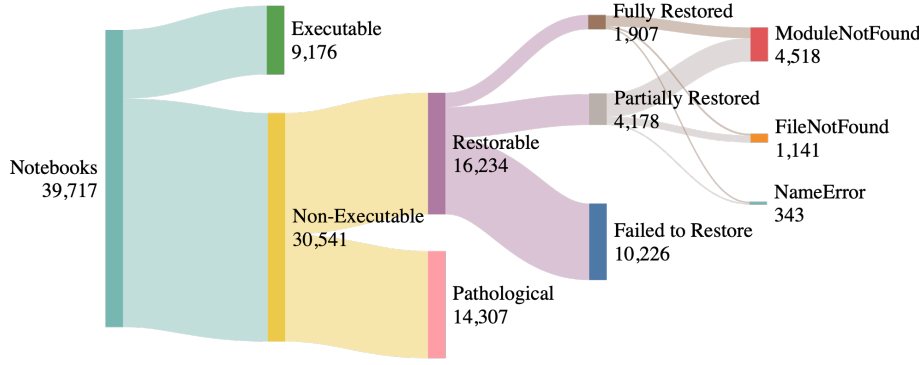


Fig. 6 Summary of results illustrating various notions of executability in computational notebooks

Study 1, a missing input data file may trigger a runtime error, rendering the notebook non-executable. However, providing the correct execution environment and data file enables the notebook to run successfully without altering its semantics.

We identify 23.1% (9,176 / 39,717) notebooks initially executable, indicating end-to-end execution without errors. Among the rest, shown in Figure 6, we find 14,307 (46.8% or 36% of the dataset) pathologically non-executable notebooks, distinguished by errors that significantly impede their execution and require additional specification about the intended goal of the notebook to recover. 16,234 (53.2%) non-executable notebooks fall into the restorable category, suggesting errors that could be remedied through appropriate restoration strategies.

Takeaway RQ2. Surprisingly, only 36% of all notebooks are pathologically non-executable; the rest are either executable or potentially executable given a suitable execution environment. This validates our hypothesis that pathologically non-executable notebooks are significantly lower than previously found.

5.3 RQ3: Degree of Executability of Pathologically Non-executable Notebooks

Notebooks are developed and executed incrementally and interactively, unlike traditional software, which is built and run atomically. Even if a notebook is pathologically non-executable as a whole, it may still contain valuable code snippets that are executable. This introduces a fine-grained notion of executability, where errors may occur only in cells toward the later stages of the notebook. Our position is that these executable cells provide valuable code fragments that can be used for dynamic analysis and supporting tasks such as code reuse, comprehension, repair, and model training. With this new, fine-grained executability, we measure the number of cells a pathologically non-executable notebook successfully executes. We then divide this number by the total number of cells in the notebooks to measure the degree of executability (partial execution) in pathologically non-executable notebooks.

Our analysis reveals that, on average, 31.3% of the cells in pathologically non-executable notebooks are executable. For example, although the notebook (jardHAN 2019) encounters “ValueError” during linear execution, it is still partially

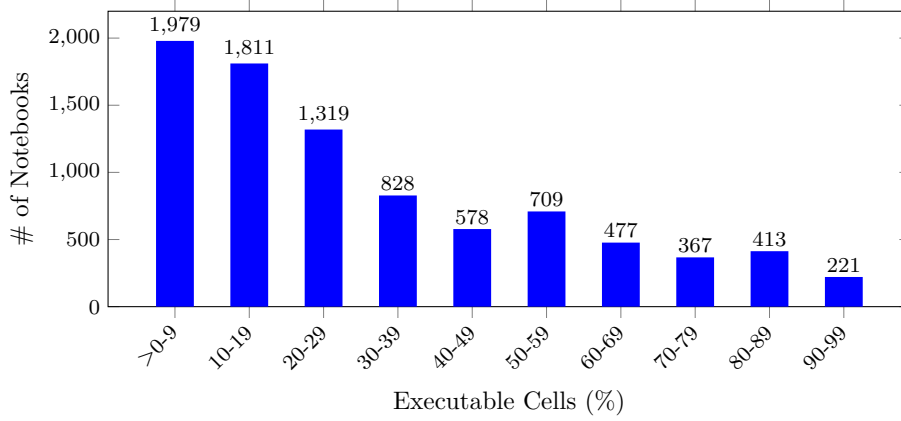


Fig. 7 Executability of pathologically non-executable notebooks measured by percentage of successfully executed cells

executable up to 10 cells or in 76.9% of total cells. Figure 7 shows the frequency of pathologically non-executable notebooks with different levels of partial executability. The Y-axis represents the number of notebooks, and the X-axis represents the percentage of cells successfully executed in those notebooks. For instance, we observe that nearly 12.8% of the pathologically non-executable notebooks (1,829) successfully execute more than half of the cells (>50%). On the other hand, 5,583 (39%) notebooks have 0% partial executability due to the pathological error in the first cell. Partial execution in nearly three-quarters of pathological non-executable notebooks demonstrates that even non-executable notebooks can be partially adopted, and dynamic analysis techniques can be applied to improve their executability and reproducibility further.

Takeaway RQ3. Even pathologically non-executable notebooks contain, on average, 31.3% executable cells. This demonstrates that partially executable notebooks hold valuable code fragments for downstream restoration steps.

5.4 RQ4: LLM-Based Error-Driven Restoration

After executing notebooks (Section 4.1.2), we categorize 30,541 non-executable notebooks into two main categories: 14,307 pathologically non-executable notebooks and 16,234 restorable notebooks (see Figure 6). They account for 46.8% and 53.2% of total non-executable notebooks, respectively. This section analyzes the executability of restorable notebooks when restored using lightweight LLM-based error-driven strategies.

5.4.1 Addressing *ModuleNotFound*

Our study reveals that 12,430 notebooks encountered “ModuleNotFound” errors out of 16,234 notebooks, accounting for 76.6% of the restorable notebooks. We use an

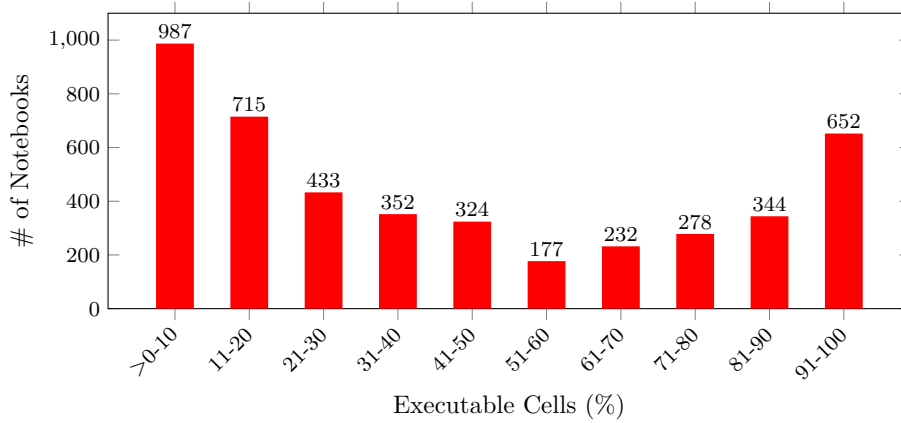


Fig. 8 Improvement in executability by resolving ModuleNotFound errors

error-driven, code-aware prompt to request the LLM to infer the missing module name, and then we package the missing module name to install the correct environment for each notebook. During this restoration process, we discover that 57 (0.5%) encounter “ModuleNotFound” errors due to deprecated modules, libraries, and packages (Wang et al. 2021; Zhu et al. 2021), changes in installation procedures, or missing custom packages. Old versions of certain packages are sometimes removed from `pip` repositories due to security vulnerabilities, licensing issues, or to encourage the use of newer, stable releases. This removal makes these versions unavailable, preventing us from resolving dependencies.

Among the restorable notebook that imports a valid public module, we successfully installed the required modules into the execution environment for 4,518 (resulting in 36.5% success rate), while 7,855 failed to do so. Module installation failures occur because only 43.9% of the repositories include an environment requirement file, leading to potential dependency conflicts due to unspecified module versions. For the notebooks where modules were successfully installed, the overall notebook executability increased by 43%, on average. Among notebooks with valid modules and resolved missing dependencies, 31.6% were fully restored, and 68.4% were partially restored. This improvement underscores the importance of adapting to changes in module availability and installation processes to enhance the executability of notebooks.

Figure 8 shows the number of notebooks and the corresponding improvements in executability after applying this strategy. The Y-axis represents the number of restored notebooks, and the X-axis represents the increase in executable cells. The figure shows that 652 notebooks have >90% improvements in executability. For example, the notebook (Visualize-ML 2024) initially encounters a “ModuleNotFound” error due to the absence of the `networkx` module in the environment in cell 1. The environment requirement information is not provided in the repository. This approach restores the full executability of that notebook.

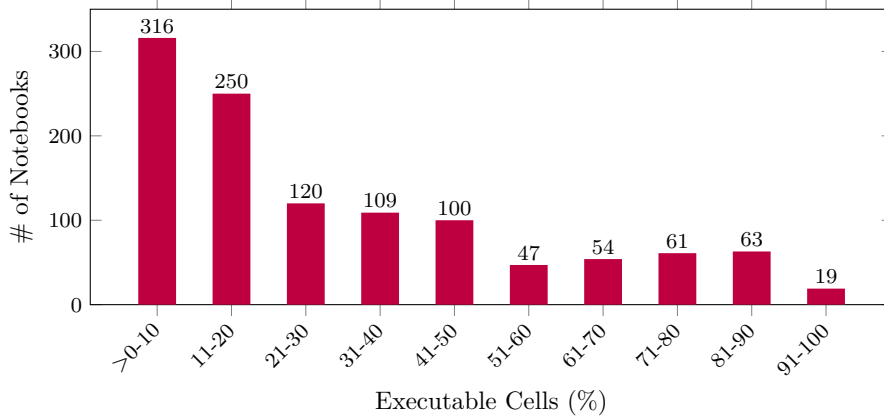


Fig. 9 Improvement in executability with synthetic input

5.4.2 Addressing *FileNotFound*

Most of the notebooks rely on input data files, which, in most cases, are not packaged with the notebook due to privacy concerns about the data or size. We find that 3,957 popular notebooks encounter “FileNotFound” errors, accounting for 24.4% of non-executable but restorable notebooks. To restore the executability of such notebooks or reveal other execution errors, we use LLM to generate synthetic data for the missing input files automatically. As mentioned in Section 4, we do not expect synthetic data to reproduce the notebook results. Instead, it can help validate the dynamic behavior of a notebook in order to understand and analyze it better.

Among the notebooks with “FileNotFound” errors, we successfully generate input files via LLMs for 2,521 (63.7%) notebooks. The generated inputs are then placed in a directory structure mentioned in the notebook automatically, and the notebook is executed again. In 2,521 notebooks that were rerun with the synthetic input files, the executability of 1,141 (45.3%) notebooks is fully or partially restored.

Our dataset of notebooks performs a wide range of tasks (statistics, ML, AI, and data analysis) across a wide range of domains, which leads to a large variety of input files being processed by these notebooks. Figure 1 shows a real-world scenario where a notebook is fully restored after generating a synthetic input file `CAR DETAILS FROM CAR DEKHO (1).csv`. The complexity of the content of this file (e.g., textual data as column names, numerical data as column values, precision in generating binary labels for the `selling_price` column, and avoiding alphabetic words in numerical columns) is shown in Figure 4. Thus, 45.3% increase in notebook execution with synthetic input files is significant and a promising direction to improve notebook executability (Section 7).

Figure 9 shows the enhancement in executability achieved in non-executable notebooks with LLM-based input generation. The X-axis represents the percentage of the increase in successfully executed cells. The Y-axis is the corresponding number of notebooks restored. For instance, 19 notebooks see more than 90% increase in executability, whereas 122 show less than 5%. Among notebooks for which we were able to generate the synthetic input files, we partially restore 36.4% of the notebooks and

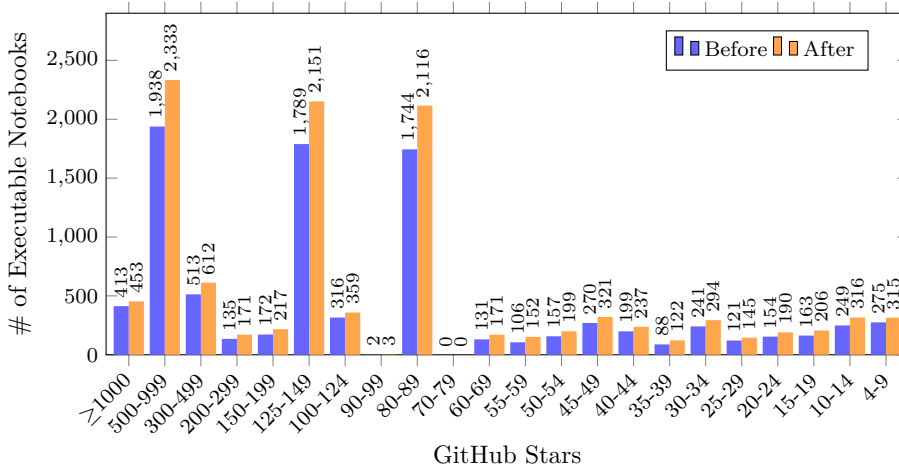


Fig. 10 Fully executable notebooks before and after restoration

fully recover 8.8% of them using the LLM-based input generation. On average, we observe an incremental improvement of 30.9% in the number of executable cells in a notebook after employing the LLM-based restoration.

5.4.3 Addressing NameError

We find only 1,269 notebook encountering “NameError,” among which only 202 had define-after-use issues. Using LLM to address this issue in a few notebooks will not offer statistically significant insights. Instead, we use LLM to assist in generating proper definitions for the undefined variables or names. Among 1,269, we enhance the executability of 343 notebooks (27%) by applying this approach. This comprises 13.8% and 13.2% notebooks fully and partially restored in the same order, out of all notebooks with “NameErrors”. Overall, this restoration leads to an average executability improvement of 42.6% across notebooks with “NameError”. Case Study 2 in Section 2.2 provides an example of a notebook facing “NameError” due to an undefined function, which prevents full execution. We use LLM to identify a definition for this function under the `tensorflow` module, improving executability by 50%. This increase is an indication of LLM’s potential for restoring functionality in notebooks with missing definitions.

Takeaway RQ4. Utilizing LLM-based error-driven restoration strategies can potentially enhance the executability of restorable notebooks. For notebooks encountering errors such as “ModuleNotFound,” “FileNotFound,” and “NameError,” we achieve improvements of 43%, 30.9%, and 42.6% in executability for each respective exception.

Finally, we summarize the impact of restoration in Figure 10, which shows the increase in the number of notebooks that achieved full executability before and after all restorations have been applied. Note that many notebooks contain multiple er-

rors, so the numbers reported for full and partial restoration from each error type may overlap and contribute to the overall restoration.

6 Automated Executability Test Generation

To address **RQ5**, we leverage our empirical findings and develop an automated executability test generation tool that ensures that notebooks are ready to use before they are made public. Since notebook ecosystems do not provide continuous integration/continuous testing (CI/CT), our auto-generated tests can serve as a substitute, helping authors catch executability issues early (Biswas et al. 2022). Note that we design this tool to run right before the notebook developer is ready to share or publish the notebooks. Unlike our restoration approach (Section 4), which sequentially executes and restores notebooks, halting at the first error encountered, the test generation approach combines both static and dynamic analyses to uncover additional issues that might remain hidden in a purely sequential execution.

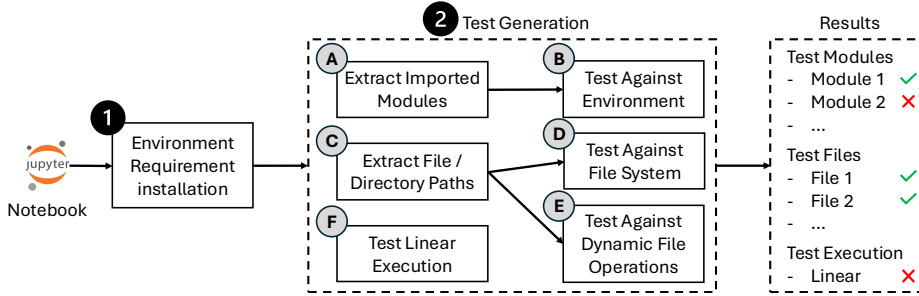


Fig. 11 Workflow of the automated test generation tool

From the empirical study findings presented earlier, we identify the top three common developer behaviors that cause notebooks to be non-executable: missing environment requirements, missing input files, and undefined variables or disordered cell execution. For each notebook, we construct test cases to verify the notebook’s ability to reproduce environment dependencies, external files, and execute cells in a linear order (see Figure 11).

Required Modules. As mentioned in Section 4, most repositories provide insufficient information about the requirements of the execution environment. This leads to “ModuleNotFound” errors being the most common error encountered in computational notebooks. In addition to reestablishing the original environment, we want to ensure that notebooks can be successfully executed in a different environment. We perform the following tests. First, we retrieve all requirement files from the repository and install the necessary dependencies in the execution environment before executing or testing any notebook (step 1 of Figure 11). This ensures that all external dependencies are available and that failed module detections accurately reflect genuine missing packages, rather than environmental inconsistencies.

Next, to analyze external dependencies, we scan the source code of all code cells to extract imported modules (2-A of Figure 11). It accounts for both absolute and

relative imports of base modules (e.g., `import numpy`) and also detects absolute imports of submodules, mapping them back to their base modules (e.g., `import numpy.linalg`). We exclude standard libraries and the built-in Python modules, and also filter out local modules present in the notebook repository to avoid false positives. Modules that are not built-in, standard, or local are considered external dependencies. This results in a set of base modules, the presence of which must be tested.

Third, for each external module detected, we generate a unit test to determine whether the module is installed in the execution environment and, if possible, retrieve its version (②-B of Figure 11). This is implemented using Python’s `importlib` utilities. Specifically, we first check whether the module can be found using the `find_spec()` function. If the module is not found, it is recorded with a status of *failed* and no version. Otherwise, the module is marked as *success*, and we attempt to retrieve its version using the `version()` function; if the version cannot be determined, it is recorded as unknown. The resulting information for each module is stored in a dictionary that maps the module name to its availability status and version.

In addition, we aggregate all tested modules across notebooks and export a cumulative `requirements.txt` file for the target repository. This file captures the set of dependencies required by the generated tests and can be used by users to reconstruct the execution environment and improve notebook executability.

To illustrate this process, consider notebook `embeddings-for-nlp.ipynb` from (Debnath 2021). We automatically extract the external modules `torch` and `tiktoken`. For each module, we generate a test case to verify its availability in the execution environment and, if possible, retrieve its version. Although the repository includes a `requirements.txt` file, this file does not list those modules. As a result, even after we install all listed dependencies, the imported modules remain unavailable in the execution environment, leading to two failed test cases. These failing test cases alert the developer to possible execution issues in the notebooks, even if the notebook runs completely in the developer’s environment.

External Data Dependency. As observed in Table 1 and previous studies, “FileNotFound” errors contribute to 10% of total notebooks, making it the third most common error in computational notebooks (Pimentel et al. 2019). To capture and test such dependencies, the naive approach involves using static analysis with regular expressions or abstract syntax trees to manually extract the paths. However, static analysis provides only partial insight. File paths can be dynamically constructed and destructed, hidden behind library abstractions, or constructed on a variable basis. To overcome these limitations, we adopt a dynamic analysis approach that systematically captures all possible user file operations as notebooks execute.

The core idea of our approach is to intercept, monitor, and record file paths and their operations at run-time. We sequentially execute each notebook cell using IPython and temporarily redirect the most common file-handling functions from the following modules using the monkey patching technique: `os`, `shutil`, `pandas`, `NumPy`, `pathlib`, and `joblib`. We then log every file read, write, creation, or deletion event (②-C of Figure 11). This includes not only direct function calls (such as `open` or `read` files) but also higher-level library routines (such as data loading and saving through `pandas` or `NumPy`) that internally rely on these mechanisms. By wrapping these operations at the appropriate abstraction level, we ensure that even indirect or library-mediated file access is observed without altering the notebook’s behav-

ior. Additionally, we utilize and customize the `VirtualFile` class to simulate path creation or deletion, thereby avoiding any actual action.

For each event, we record the access path along with its mode and location. This design allows us to distinguish between input dependencies (files read), output artifacts (files written), and destructive operations (files deleted or overwritten). Capturing this information ensures that the log reflects the actual execution trace rather than the statically assumed behaviors. We then create test cases for all possible file or directory paths that may be encountered. Initially, we check if a file or directory is provided within the repository (②-D of Figure 11). If it is not available within the file system, we then verify if it was dynamically created in the previous cells (②-E of Figure 11). If a path is neither present in the repository file system nor is being read or deleted without being created first, we categorize it as an *unfound* path. In some rare cases, a path is considered *invalid* when it is deleted more than once consecutively, meaning it becomes unavailable after the first deletion. Otherwise, we record a path as *found*. Once the analysis is complete, we restore the original file-handling behaviors, ensuring isolation and preventing interference with other processes.

Take notebook “40_extra_cut_material.ipynb” (Co. 2025), for instance. We execute the notebook cell by cell, intercepting every file operation through a patched version of the underlying function. This redirection allows us to both simulate the operation and capture its access mode. For example, when the file “`../dataset/s/students.csv`” is opened by the `pandas` function `read_csv`, we record the file path (converting relative paths to absolute ones with respect to the repository root) together with its operation mode, in this case *read*. Each distinct combination of path and operation is stored in order, forming the aggregated results that support the testing phase. After this step, we systematically test each recorded file path and its associated operation. In this example, the input file is present in the repository file system, so the corresponding test is marked as *success*.

Linear Execution. As part of the test generation, we generate a test to evaluate the full executability of each notebook. This test consists of a sequential dynamic execution of all code cells using `papermill` (nteract contributors 2018) (②-F of Figure 11). In other words, all code cells are executed as-is in their original top-to-bottom order, thereby mimicking a clean run of the notebook. The dynamic aspect refers to executing the notebook in a fresh environment, with required dependencies installed, rather than relying on pre-existing outputs or execution history. We adopt `papermill` because it provides a controlled interface for executing notebooks, such as timeout management, kernel specification, and consistent error reporting. The same tool is used in our restoration workflow, which allows us to produce directly comparable results between the testing and restoration phases. We also set the execution time to 5 minutes, which is consistent with the execution step in the restoration process. In this test, a notebook is marked as *success* if it can be completely executed without errors within the given time limit. Otherwise, the test case is considered *failed*.

6.1 Evaluation

We first measure the effectiveness of the auto-generated test cases in identifying the execution issues in notebooks by comparing the test failure and passing rates

on executable notebooks versus non-executable notebooks. Second, we generate and run auto-generated tests to measure notebook execution in the wild.

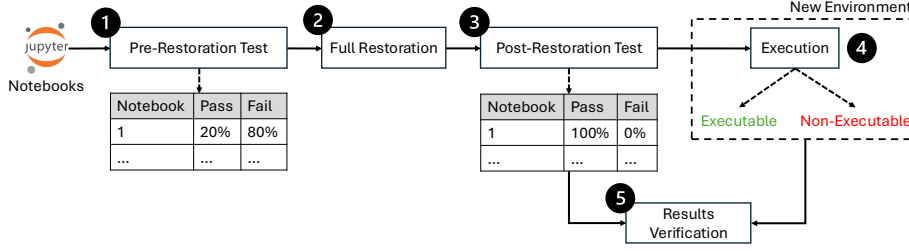


Fig. 12 Overall workflow of the evaluation of the test generation tool

Dataset. We collect an additional dataset of public notebooks with commits from January 2025 onward. We stratified this new dataset based on repository popularity, i.e., star count. Each star range comprises approximately 1000 repositories, each with a last commit on or after January 1, 2025, at the time of collection to ensure recency and relevance. We randomly take 1% from each range, resulting in more than 200 repositories. After performing an initial check (described in Section 4.1), we compiled a dataset of 192 public GitHub repositories with a total of 3,169 notebooks, covering a variety of topics and project sizes, providing a representative sample of contemporary computational notebooks on GitHub.

6.1.1 Effectiveness of Generated Test Cases in Finding Non-Executability

We evaluate the adequacy of the generated test on notebooks that are initially non-executable but become fully executable after the restoration process. We verify this by executing the notebooks, as stated in Section 4. Figure 12 illustrates the evaluation workflow. In step ❶, we generate test cases to assess the executability of notebooks before restoration. In step ❷, we apply the restoration workflow to attempt to recover the non-executable notebooks. Step ❸ reruns the test generation tool on the restored notebooks to produce a new set of test cases. Next, in step ❹, we execute each fully restored notebook in a clean environment that includes all required dependencies, input files, and variable definitions (if any of these were missing before restoration). This step represents that the notebook is reused in a different environment. Finally, step ❺ compares the post-restoration test results with the actual execution outcomes to verify whether the test cases correctly detect executability.

Figure 13 presents the distribution of the notebooks by the percentage of passed tests before and after restoration. Before restoration, the notebooks achieve, on average, 32.5% passing tests. After restoration, this increases by 67.5%, resulting in an average of 100% passing tests. Notably, all 131 notebooks reach 100% passing tests after restoration. For example, the notebook `stats_ch13_confidenceIntervals.ipynb` from (Cohen 2025) achieves only 50% passing test cases prior to execution and restoration. After fully restoring all “ModuleNotFound” errors caused by missing dependency information in the absent `requirements.txt` file, the notebook reaches a 100% passing rate. This correlation between increasing passing test rates when the

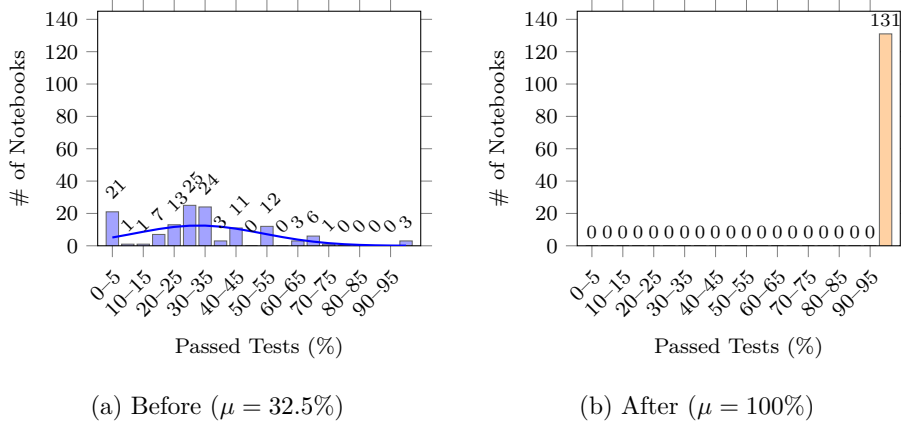


Fig. 13 Distributions of passed tests percentage (per notebook) before and after full restoration

executability of notebooks increases reveals a strong correlation, indicating that our generated tests are capable of identifying non-executable notebooks and systematically uncovering the factors that cause non-executability.

6.1.2 Executability Issues in the Wild Using Automated Executability Tests

In Section 4, we restore notebook executability by dynamically executing them and addressing errors as they occur, halting at the first failure encountered. Consequently, many errors that appear in later cells may remain hidden. However, with automatically generated executability tests, we can reveal these hidden issues, providing a more comprehensive assessment of notebook executability before full execution. We generate tests on the entire dataset, collected from 2025, regardless of their pathological executability, to systematically evaluate the executability of public notebooks and identify factors that could impede their successful execution.

Figure 14 shows the distribution of the notebooks based on the percentage of tests passed per notebook before restoration. On average, notebooks achieve approximately 73.1% passing tests. Among the 3,169 notebooks, 982 notebooks (31%) exhibit a passing rate greater than 95%, indicating that a substantial portion of the dataset is highly executable even before restoration. Among the three types of tests, computational notebooks most often encounter problems with environment dependencies, with 34.5% of the dataset failing at least one such test case. In comparison, the external input file and linear execution tests exhibit slightly lower failure rates, affecting 9.2% and 31% of the notebooks, respectively.

Takeaway RQ5. By applying the automated test generation tool, we produced approximately 17,000 test cases for the 2025 dataset. Our results show that the generated test cases accurately reflect notebook executability, with an average success rate of 64.8% on non-executable notebooks, compared to a much higher rate of 100% on fully executable notebooks.

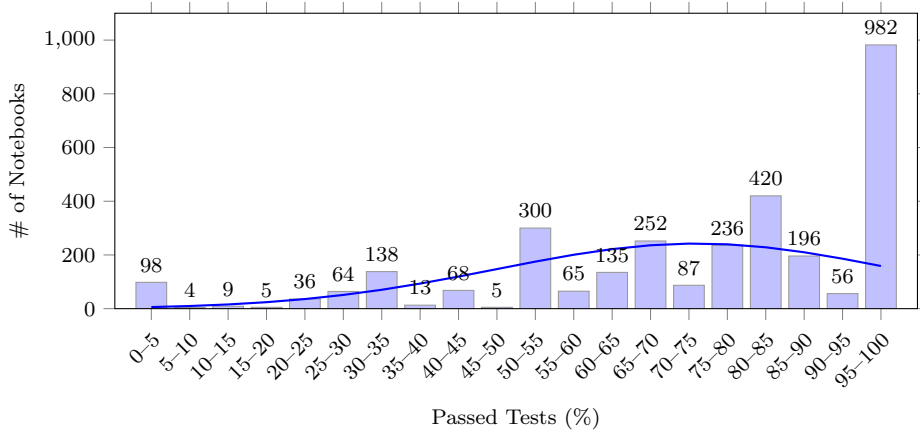


Fig. 14 Distribution of passed tests percentage (per notebook) before restoration in the 2025 dataset ($\mu = 73.1\%$)

7 Discussion

In this section, we distill key findings in a *FAQ* format, demonstrating the value of accurate executability categorization, the utility of a notebook corpus once marked unusable, and practices that could prevent non-executability or restore executability in future notebooks.

Does the quality of public repositories improve? In the 2025 dataset, 31% of notebooks were initially executable, increasing to 35.2% after restoration. This rate is higher than in the 2024 dataset, where only 23.1% were initially executable and 27.9% after restoration. The 2025 dataset is a subset of the 2024 collection, restricted to repositories with the latest commits in 2025, indicating that these repositories are actively maintained. Consequently, they exhibit better executability than previously reported. The improvement likely reflects differences in dataset composition: newer repositories tend to be more popular, better maintained, and more likely to include explicit dependency files, whereas older collections contained many unmanaged or experimental notebooks. For example, several 2025 repositories included multiple requirement files tailored for different processes, which enabled full restoration—unlike many 2024 notebooks that lacked such specifications. These findings suggest a gradual trend toward higher repository quality, consistent with the growing availability of notebook testing and validation tools discussed later in Section 9.4.

Why are 36% notebooks pathologically non-executable and 25.3% of restorable notebooks still only partially executable? Prior work has identified that notebooks have alarmingly low numbers of test cases (Pimentel et al. 2019). Developers releasing their notebooks for public use have very limited testing tools to verify reproducibility and executability. For example, notebooks with undefined variables “NameError” and “AttributeError” often go unnoticed due to unintended dependencies on session states saved from prior cell execution, leading to those errors being masked on the developer’s machine, similar to Case Study 2 in Section 2.2. In a new environment, such states are unavailable, resulting in these errors. Our findings on pathological

errors in notebooks demonstrate the need for static and dynamic analysis tools to catch such errors early in notebook development.

How can LLMs further restore notebook executability with a high success rate? In the first case study (Section 2.1), Gemma3 successfully generated synthetic data for a ‘.csv’ file. However, in another case (Sarkar 2019), it failed to produce a valid ‘PNG’ file due to limitations in multi-modal generation. This is because most LLMs have been trained on similar textual data formats. Therefore, Gemma3 also faces limitations when generating images, audio, video, or zip files. Current multi-modal models, such as GPT-5 or Gemini 2.5 Flash, can generate richer and more diverse inputs. This capability can benefit developers who want to share their notebooks without disclosing the input data files. They can leverage LLMs to generate synthetic data (or scripts for synthetic data generation similar to database benchmarks (Transaction Processing Performance Council 2008)), enhancing executability in remote environments. Additionally, LLM performance tends to degrade when provided with large contexts, which aligns with recent research on LLMs (Leng et al. 2024). Moreover, notebook executability can be enhanced by generating feedback-driven fixes (e.g., multi-shot) for non-executable cells. This includes generating new input files based on error feedback.

What use is there for partially executable notebooks? Prolonging notebook execution leads to more cells being executed successfully, which translates into more code surfaces that can be executed and understood. This inherently improves code reuse and enhances the chances of notebooks being fully restored. Furthermore, dynamic analysis techniques are limited to executable code only. By improving notebook executability, we increase the application surface of dynamic analysis tools such as dynamic taint analysis (Clause et al. 2007), runtime tracing (Meliou et al. 2011), automated debugging (Parnin and Orso 2011), symbolic execution (Baldoni et al. 2018), and Osiris (Wang et al. 2020) that can play vital roles in recovering notebook reproducibility. Partial executability metrics can also provide fine-grained measures of the effectiveness of reproducibility and execution-restoration tools. Additionally, it can serve as valuable feedback when applying code repair techniques in notebooks, such as automated cell reordering (Wang et al. 2020).

How do the findings of this study improve notebooks? We explored several design choices in building our measurement framework. We identify gaps in the notebook ecosystem for code review, debugging, test generation, and build tools. For existing public notebooks, which are growing exponentially (Rule et al. 2018b), this measurement framework can be extended to improve fine-grained executability, feeding into dynamic analysis tools to enhance reproducibility.

Our findings, showing that 1,446 (43.9%) repositories have REQUIREMENTS files, indicate that there is no standardized build and continuous integration mechanism for notebooks. While this may not apply to exploratory, standalone, one-off notebooks, the most popular repositories provide a range of notebooks that must be properly orchestrated to ensure executability. We believe Python-based build tools, such as `disutils` (Foundation 2024) and `setuptools` (Authority 2025), can be extended to support notebook extensions at the browser level to enforce correct build and dependency management for notebooks.

8 Threats to Validity

Dataset Curation and Execution Constraints. Outdated, textual, and empty notebooks can cause variations in the results of this study. To mitigate that, we exclude notebooks written in Python 2 since support for Python 2 was discontinued in 2020. We also filter empty or instructional notebooks from our dataset. Similarly, focusing purely on GitHub repositories can bias the results towards a specific class of notebooks. Upon investigation, we identified numerous HuggingFace and Kaggle notebooks that are also hosted on GitHub. While there is always room to increase the scale of the analysis, our emphasis on popular, actively reusable notebooks deprioritizes the need to expand the scale. Our static and dynamic analyses rely on the Python AST parser and Python interpreter to capture errors. These tools can potentially miss or misclassify errors, which can impact our categorization. Since most of our runtime errors are captured through dynamic error checking, an earlier fatal runtime error (that we cannot resolve) may hinder our ability to capture “FileNotFoundError” or “ModuleNotFoundError” errors. Lastly, we use a timeout of 5 minutes to execute each notebook. This approach may exclude notebooks like the ones requiring expensive machine learning training. However, in our notebook corpus, only 2.4% of notebooks encountered timeout errors.

File I/O Library Coverage. Our current testing generation implementation focuses on a subset of commonly used file-processing libraries (e.g., `pandas`) when detecting file operations. While this may not cover all libraries used in practice, our design is lightweight and can be extended to additional libraries with minimal effort. As an example, we extended the implementation to support `joblib`, requiring only 47 lines of code and approximately 30 minutes of implementation effort. This demonstrates that the approach can be incrementally expanded to support additional libraries without major changes to the system design.

Sensitivity to LLM Choice. We acknowledge that relying on a single LLM as an evaluation component provides only a point estimate and does not capture variability across models, which may limit the generalizability of our findings. To mitigate this, we conducted additional experiments on a representative subset of the dataset, evenly sampled across star tiers, using Gemma 4 (26B parameters) (Google 2026). This allows us to assess the sensitivity of our evaluation pipeline to the choice of LLM.

We evaluated a total of 113 repositories, containing 978 notebooks. The results with Gemma4 show similar overall trends compared to Gemma3 used in Section 5. In particular, 23.3% and 76.7% of notebooks are initially executable, compared to 23.1% and 76.9% reported in the paper. For non-executable notebooks, the LLM-based restoration approach fully restores 7.9% and partially restores 14.8% of them, resulting in 85.7% and 22.2% of executability increase, on average, respectively. For “FileNotFoundError”, “ModuleNotFoundError”, and “NameError”, the LLM-driven restoration approach improves an average of 24.9%, 41.6%, and 22% of notebook executability, in the same order. Overall, Gemma4 achieves slightly better restoration performance than Gemma3. This improvement is likely due to its Mixture-of-Experts (MoE) architecture, which provides stronger reasoning capabilities in math and coding tasks. Since our original experiments used a more conservative lower-bound model, these results suggest that more advanced LLMs may further strengthen our findings.

Limitations of LLM-Generated Data Artifacts. Our approach relies on an LLM to generate missing input artifacts required for notebook execution. While this includes

Table 3 Comparison of some related works with ours in the computational notebook topic

	Purpose	Error Types			Non-Exec*	Study Dataset	
		Module	Name	File		#	Characteristics
a	Executability	✓	✗	✗	47%	4,043	Collected from Meta Kaggle (Kaggle n.d.)
b	Executability	✓	✗	✗	72.6%	2,646	Random sample from (Pimentel et al. 2019) only those with installable dependency
c	Reproducibility	✗	✓	✗	82.6%	5,393	Random sample from (Pimentel et al. 2019)
d	Empirical	N/A	N/A	N/A	76%	>1.4M	Includes high-number of low-starred, rarely reused notebooks from GitHub
This work		✓	✓	✓	36%	39.7K	Highly-starred notebooks from GitHub

^a RELANCER (Zhu et al. 2021)^b SnifferDog (Wang et al. 2021)^c Osiris (Wang et al. 2020)^d Pimentel et al. (2019; 2021)

* Non-executable notebooks found

structured formats such as JSON and plain text, LLMs remain limited in generating complex or domain-specific non-tabular data, such as images or multimodal datasets, where outputs may not faithfully reflect the original data distribution or semantics. However, our goal is executability rather than data fidelity, and notebooks can still execute as long as functionally compatible surrogate inputs are produced, even if semantic equivalence is not preserved. This introduces a potential threat to validity for workloads that depend on high-fidelity multimodal data. Overall, we consider this a conservative setting, as we use a standard open-source LLM as a lower bound, and stronger or multimodal models may further improve artifact quality and strengthen the observed executability trends.

9 Related Work

With the increasing popularity of notebooks (Kluyver et al. 2016), a growing body of research has focused on understanding the unique characteristics of notebooks (Chattopadhyay et al. 2020; In et al. 2024; Rule et al. 2018a,b; Samuel and Mietchen 2024) and their diverse applications in various fields (Li et al. 2021; Randles et al. 2017; Wang et al. 2019a). Recent works have also proposed potential enhancements to these tools, aiming to improve notebooks’ functionality and usability for data scientists and other users (Chattopadhyay et al. 2020; McNutt et al. 2023; Samuel and Mietchen 2024; Wang et al. 2024b). Table 3 summarizes and compares this work with prior investigations.

9.1 Empirical Studies on Notebook Executability

Pimental et al. (2019; 2021) conducted an analysis of 1.4 million notebooks sourced from GitHub, aiming to examine their nature, quality, and reproducibility. They report notebook-quality results, covering various characteristics and insights into notebook execution processes, as well as common issues encountered within these notebooks. Similarly, Wang et al. (2019b) assesses the quality of code present in notebooks, ultimately concluding that notebooks often exhibit suboptimal coding practices, thus highlighting the importance of enhancing notebook code quality. Both studies only view notebook executability as an atomic measure. In particular, they lack a fine-grained, deeper analysis of a large portion of notebooks that are (1) non-executable only on non-author environments due to misconfiguration, thus fully restorable, and (2) offer reusable, valuable code that is partially executable. Wang et al. (2025) conducted an empirical study of machine learning notebooks and identified that many failures arise from improper API usage and notebook-specific pitfalls.

9.2 Notebook Restoration and Reproducibility Tools

Numerous tools are proposed to restore notebook executability and reproducibility. For instance, RELANCER automatically updates deprecated APIs in non-executable notebooks by gathering APIs from GitHub and documentation (Zhu et al. 2021). Similarly, SnifferDog restores the execution environment by providing a collection of APIs from Python packages and libraries to update required packages in notebooks (Wang et al. 2021). These approaches complement our analysis. However, due to the highly complex and heavyweight nature of these techniques, they are infeasible for large-scale notebook executability analysis. Osiris restores the reproducibility of only fully executable notebooks that also contain output cells (Wang et al. 2020). It addresses cell dependencies and generates possible execution orders, similar to our def-use set-based reordering. Osiris demonstrates that without executable notebooks, reproducibility can not feasibly be restored. Similar to our investigation, a transparent, fine-grained view of the state of notebook executability can facilitate further reproducibility studies or tool development.

9.3 Notebook Development Assistance Tools

Previous research has proposed improving software development practices with extended user interfaces and advanced programming assistant features. Fork It is a forking and backtracking extension designed to investigate various alternatives and traverse different states within a single notebook (Weinman et al. 2021). Similarly, numerous notebook tools (Li et al. 2024; Rule et al. 2018a; Wang et al. 2024a,c; Zhu et al. 2024) aim to streamline collaborative and exploratory experiments, often incorporating new visualization and conflict resolution for collaborative work in notebooks. To assist in code debugging and cleaning, Robinson et al. (2022) examined error identification strategies used for Python notebooks, while Head et al. (2019) utilized program slicing to extract relevant code cells producing specific outputs. In addition, HeaderGen uses static analysis and a machine-learning-based taxonomy

to automatically generate and classify headers for Jupyter notebook code cells, improving comprehension and navigation (Shivarpatna Venkatesh et al. 2023). Adding new features and complicated user interfaces significantly intervenes with rapid, interactive programming. Most of these tools have seen resistance in adoption by both users and popular notebook platform providers. With the emergence of LLMs, recent work has explored using LLMs in assisting notebook development (Grotov et al. 2024; McNutt et al. 2023; Wang et al. 2024b; Weber and Mayer 2024). These results resonate with our findings.

9.4 Notebook Testing Tools

Users often use notebooks without ensuring code quality; hence, public notebooks usually have numerous issues and are not well tested (De Santana et al. 2024). Prior studies on testing Jupyter notebooks have mainly concentrated on content-based validation. For example, existing approaches introduce unit tests that check the correctness of code cells and outputs (Fangohr et al. 2020; Sun 2024). These approaches aim to verify whether the expected outputs are produced given specific inputs. Pynblint statically analyzes notebooks for best practices and style issues but does not execute them, and thus cannot verify their executability or reproducibility (Quaranta et al. 2022b). While these methods improve confidence in localized results, they fall short in ensuring that notebooks can be executed end-to-end in a reproducible manner. Our work extends this line of research by shifting the focus toward executability and reproducibility. Oli et al. (2021) employed machine learning, using both expert-defined features and semantic code embeddings, to automatically predict the executability and reproducibility of Jupyter notebooks; however, their approach does not provide insight into the underlying causes of non-executability.

10 Conclusion

Computational notebooks, widely adopted in data science and ML/AI workflows, remain prone to executability challenges. Prior studies often rely on rigid executability definitions, which overestimate the prevalence of non-executable notebooks. In this paper, we are the first to introduce the concepts of *partial executability* and *pathological non-executability*, providing a more realistic lens on notebook execution. Our empirical analysis shows that 36% out of 39,717 notebooks are pathologically non-executable, while 16,234 can be restored under appropriate environments. Using LLM-based error-driven restoration, we fully restore 11.7% and partially restore 25.3% of these previously non-executable but restorable notebooks. Building on these findings, we designed an automated executability test generation tool, which systematically detects hidden executability issues and evaluates notebook quality. Its application to the newly collected dataset reveals both persistent challenges and opportunities for improving executability. Our results demonstrate that restoration techniques and automated testing can significantly enhance notebook reusability and reproducibility.

Acknowledgements We thank the Advanced Research Computing Center at Virginia Tech for their support in building and evaluating this work.

Declarations

Funding

This work was supported in part by Amazon - Virginia Tech Initiative in Efficient and Robust Machine Learning, 4-VA, and the National Science Foundation award 2106420.

Ethical approval

Not applicable.

Informed consent

Not applicable. This research does not involve human participants and/or animals.

Author Contributions

Conceptualization: Tien Nguyen, Muhammad Ali Gulzar; *Methodology:* Tien Nguyen; *Software:* Tien Nguyen, Waris Gill; *Formal analysis and investigation:* Tien Nguyen; *Writing – original draft preparation:* Tien Nguyen; *Writing – review and editing:* Tien Nguyen, Waris Gill, Muhammad Ali Gulzar; *Funding acquisition:* Muhammad Ali Gulzar; *Resources:* Muhammad Ali Gulzar; *Supervision:* Muhammad Ali Gulzar.

Data Availability Statement

We have made our code and dataset publicly available at <https://github.com/renote2024/renoteTest>.

Conflict of Interest

Muhammad Ali Gulzar has an institutional conflict of interest with Amazon, as he holds the role of Amazon Scholar. The authors declare that they have no conflict of interest beyond the standard institutional, co-authorship, and advisor-advisee COIs.

Clinical Trial Number

Not applicable

References

- Atienza G (2018) URL <https://github.com/gemaatienza/Deep-Learning-Course/blob/master/5.%20Sequence%20Models/Dinosaur%20Island%20--%20Character%20level%20language%20model%20final%20-%20v3.ipynb>, accessed: 2024-11-09
- Authority PP (2025) Building and distributing packages with setuptools - setuptools 80.9.0. URL <https://setuptools.pypa.io/en/latest/userguide/>
- Baldoni R, Coppa E, D’elia DC, Demetrescu C, Finocchi I (2018) A survey of symbolic execution techniques. *ACM Computing Surveys (CSUR)* 51(3):1–39
- Biswas S, Wardat M, Rajan H (2022) The art and practice of data science pipelines: A comprehensive study of data science pipelines in theory, in-the-small, and in-the-large. In: *Proceedings of the 44th International Conference on Software Engineering, ACM, ICSE ’22*, p 2091–2103, DOI 10.1145/3510003.3510057, URL <http://dx.doi.org/10.1145/3510003.3510057>
- Borges H, Valente M (2018) What’s in a github star? understanding repository starring practices in a social coding platform. *Journal of Systems and Software* 146:112–129, DOI 10.1016/j.jss.2018.09.016
- Chattopadhyay S, Prasad I, Henley AZ, Sarma A, Barik T (2020) What’s wrong with computational notebooks? pain points, needs, and design opportunities. *Conference on Human Factors in Computing Systems - Proceedings* DOI 10.1145/3313831.3376729
- Choudhari P (2021) URL <https://github.com/pratik-choudhari/AlgoCode/tree/master>, accessed: 2025-09-18
- Clause J, Li W, Orso A (2007) Dytan: a generic dynamic taint analysis framework. In: *Proceedings of the 2007 international symposium on Software testing and analysis*, pp 196–206
- Co M (2025) URL https://github.com/mikexcohen/Statistics_book/blob/main/python/stats_ch13_confidenceIntervals.ipynb, accessed: 2025-09-20
- Cohen MX (2025) URL https://github.com/minireference/noBSstats/blob/main/notebooks/40_extra_cut_material.ipynb, accessed: 2025-09-18
- De Santana TL, Neto PADMS, De Almeida ES, Ahmed I (2024) Bug analysis in jupyter notebook projects: An empirical study. *ACM Trans Softw Eng Methodol* 33(4), DOI 10.1145/3641539, URL <https://doi.org/10.1145/3641539>
- Debnath S (2021) URL <https://github.com/debnsuma/fcc-ai-engineering-aws/blob/main/01-embeddings/embeddings-for-nlp.ipynb>, accessed: 2025-09-18
- Deutsch LP, Berkeley EC (1964) The lisp implementation for the pdp-1 computer. URL https://s3data.computerhistory.org/pdp-1/DEC.pdp_1.1964.102650371.pdf
- Fangohr H, Fauske V, Kluyver T, Albert M, Laslett O, Cortés-Ortuño D, Beg M, Ragan-Kelly M (2020) Testing with jupyter notebooks: Notebook validation (nbval) plug-in for pytest. URL <https://arxiv.org/abs/2001.04808>, 2001.04808
- Foundation PS (2024) Distutils — building and installing python modules - python 3.10.16 documentation. URL <https://docs.python.org/3.10/library/distutils.html>
- Google (2025) Gemma3. URL <https://deepmind.google/models/gemma/gemma-3/>

- Google (2026) Gemma 4. URL <https://ollama.com/library/gemma4>
- Grotov K, Titov S, Zharov Y, Bryksin T (2024) Untangling knots: Leveraging llm for error resolution in computational notebooks. 2405.01559
- Head A, Hohman F, Barik T, Drucker SM, DeLine R (2019) Managing messes in computational notebooks. Conference on Human Factors in Computing Systems - Proceedings DOI 10.1145/3290605.3300500
- Hosni Y (2023) URL <https://github.com/youssefHosni/Practical-Machine-Learning/blob/main/How%20To%20Split%20The%20Data%20Effectively%20for%20Your%20Data%20Science%20Project.ipynb>, accessed: 2025-09-18
- Huang Y, Xu F, Zhou H, Chen X, Zhou X, Wang T (2022) Towards exploring the code reuse from stack overflow during software development. In: Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension, Association for Computing Machinery, New York, NY, USA, ICPC '22, p 548–559, DOI 10.1145/3524610.3527923, URL <https://doi.org/10.1145/3524610.3527923>
- In S, Krokos E, Whitley K, North C, Yang Y (2024) Evaluating navigation and comparison performance of computational notebooks on desktop and in virtual reality. In: Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems, Association for Computing Machinery, New York, NY, USA, CHI '24, DOI 10.1145/3613904.3642932, URL <https://doi.org/10.1145/3613904.3642932>
- JadFatTail (2020) URL https://github.com/JadFatTail/quant_struggle/blob/master/èµĎéĜŖçŕü/èµĎéĜŖçŕü.ipynb, accessed: 2024-11-09
- jarodHAN (2019) URL <https://github.com/jarodHAN/Python-100-Days-master/blob/master/Day76-90/code/8-pandasâĖŁçžĎèĀžĀŖĹæŖDä;æ.ipynb>, accessed: 2024-11-09
- Kaggle (n.d.) URL <https://www.kaggle.com/>, accessed: 2024-10-22
- Kluyver T, Ragan-Kelley B, Pérez F, Granger B, Bussonnier M, Frederic J, Kelley K, Hamrick J, Grout J, Corlay S, Ivanov P, Avila D, Abdalla S, Willing C (2016) Jupyter notebooks—a publishing format for reproducible computational workflows. Positioning and Power in Academic Publishing: Players, Agents and Agendas - Proceedings of the 20th International Conference on Electronic Publishing, ELPUB 2016 pp 87–90, DOI 10.3233/978-1-61499-649-1-87
- Leng Q, Portes J, Havens S, Zaharia M, Carbin M (2024) Long context rag performance of llms. Mosaic AI Research URL <https://www.databricks.com/blog/long-context-rag-performance-llms>
- Li X, Wang Y, Wang H, Wang Y, Zhao J (2021) Nbsearch: Semantic search and visual exploration of computational notebooks. Conference on Human Factors in Computing Systems - Proceedings DOI 10.1145/3411764.3445048
- Li Z, Chockchowwat S, Fang H, Sahu R, Thakurdesai S, Pridaphatrakun K, Park Y (2024) Demonstration of elasticnotebook: Migrating live computational notebook states. In: Companion of the 2024 International Conference on Management of Data, Association for Computing Machinery, New York, NY, USA, SIGMOD/PODS '24, p 540–543, DOI 10.1145/3626246.3654752, URL <https://doi.org/10.1145/3626246.3654752>
- Lin F, Kim DJ, Tse-Husn, Chen (2024) Soen-101: Code generation by emulating software process models using large language model agents. URL <https://arxiv.org/abs/2403.15852>, 2403.15852
- McNutt AM, Wang C, Deline RA, Drucker SM (2023) On the design of ai-powered code assistants for notebooks. Conference on Human Factors in Computing Systems - Proceedings DOI 10.1145/3544548.3580940

- Meliou A, Gatterbauer W, Nath S, Suciu D (2011) Tracing data errors with view-conditioned causality. In: Proceedings of the 2011 ACM SIGMOD International Conference on Management of data, pp 505–516
- Nejjar M, Zacharias L, Stiehle F, Weber I (2024) Llms for science: Usage for code generation and data analysis. *Journal of Software: Evolution and Process* n/a(n/a):e2723, DOI <https://doi.org/10.1002/smr.2723>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/smr.2723>, <https://onlinelibrary.wiley.com/doi/pdf/10.1002/smr.2723>
- Nguyen T, Gill W, Gulzar MA (2025) Are the majority of public computational notebooks pathologically non-executable? In: 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), pp 396–407, DOI 10.1109/MSR66628.2025.00070
- nteract contributors (2018) Papermill: Parameterizing and executing notebooks. URL <https://papermill.readthedocs.io/en/latest/>, accessed: 2026-04-17
- Oli P, Banjade R, Tamang LJ, Rus V (2021) Automated assessment of quality of jupyter notebooks using artificial intelligence and big code. *The International FLAIRS Conference Proceedings* 34, DOI 10.32473/flairs.v34i1.128560, URL <https://journals.flvc.org/FLAIRS/article/view/128560>
- Parnin C, Orso A (2011) Are automated debugging techniques actually helping programmers? In: Proceedings of the 2011 International Symposium on Software Testing and Analysis, Association for Computing Machinery, New York, NY, USA, ISSTA '11, p 199–209, DOI 10.1145/2001420.2001445, URL <https://doi.org/10.1145/2001420.2001445>
- Perez F, Granger BE (2021) Project jupyter: Computational narratives as the engine of collaborative data science. *Jupyter Blog* URL <https://blog.jupyter.org/project-jupyter-computational-narratives-as-the-engine-of-collaborative-data-science-2b3b8f106ac0>
- Pimentel JF, Murta L, Braganholo V, Freire J (2019) A large-scale study about quality and reproducibility of jupyter notebooks. *IEEE International Working Conference on Mining Software Repositories 2019-May*:507–517, DOI 10.1109/MSR.2019.00077, URL <https://dl.acm.org/doi/10.1109/MSR.2019.00077>
- Pimentel JF, Murta L, Braganholo V, Freire J (2021) Understanding and improving the quality and reproducibility of jupyter notebooks. *Empirical Software Engineering* 26:1–55, DOI 10.1007/S10664-021-09961-9/FIGURES/25, URL <https://link.springer.com/article/10.1007/s10664-021-09961-9>
- Quaranta L, Calefato F, Lanubile F (2022a) Eliciting best practices for collaboration with computational notebooks. *Proceedings of the ACM on Human-Computer Interaction* 6, DOI 10.1145/3512934
- Quaranta L, Calefato F, Lanubile F (2022b) Pynblint: a static analyzer for python jupyter notebooks. In: Proceedings of the 1st International Conference on AI Engineering: Software Engineering for AI, Association for Computing Machinery, New York, NY, USA, CAIN '22, p 48–49, DOI 10.1145/3522664.3528612, URL <https://doi.org/10.1145/3522664.3528612>
- Ramlochan S (2024) The power of concise prompts in large language models. URL <https://promptengineering.org/the-power-of-concise-prompts-in-large-language-models/>
- Randles BM, Pasquetto IV, Golshan MS, Borgman CL (2017) Using the jupyter notebook as a tool for open science: An empirical study. *Proceedings of the ACM/IEEE*

- Joint Conference on Digital Libraries DOI 10.1109/JCDL.2017.7991618
- Robinson D, Ernst NA, Vargas EL, Storey MAD (2022) Error identification strategies for python jupyter notebooks. IEEE International Conference on Program Comprehension 2022-March:253–263, DOI 10.1145/3524610.3529156
- Rule A, Drosos I, Tabard A, Hollan JD (2018a) Aiding collaborative reuse of computational notebooks with annotated cell folding. Proceedings of the ACM on Human-Computer Interaction 2, DOI 10.1145/3274419
- Rule A, Tabard A, Hollan JD (2018b) Exploration and explanation in computational notebooks. Conference on Human Factors in Computing Systems - Proceedings 2018-April, DOI 10.1145/3173574.3173606
- Samuel S, Mietchen D (2024) Computational reproducibility of Jupyter notebooks from biomedical publications. GigaScience 13:giad113, DOI 10.1093/gigascience/giad113, URL <https://doi.org/10.1093/gigascience/giad113>, https://academic.oup.com/gigascience/article-pdf/doi/10.1093/gigascience/giad113/55444780/giad113_reviewer_2_report_revision_1.pdf
- Sarkar T (2019) URL https://github.com/tirthajyoti/Stats-Maths-with-Python/blob/master/Set_Algebra_with_Python.ipynb, accessed: 2024-08-30
- Shivarpatna Venkatesh AP, Wang J, Li L, Bodden E (2023) Enhancing comprehension and navigation in jupyter notebooks with static analysis. In: 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp 391–401, DOI 10.1109/SANER56733.2023.00044
- Sun K (2024) A study of justification on jupyter notebook quality & fairness. URL <http://hdl.handle.net/11375/29641>
- Transaction Processing Performance Council (2008) Tpc-ds: The next generation decision support benchmark. TPC Benchmark DS Standard Specification URL <http://www.tpc.org/tpcds/>, version 2.0.1
- Visualize-ML (2024) URL https://github.com/Visualize-ML/Book6_First-Course-in-Data-Science/blob/main/Book6_Ch11_Python_Codes/Bk6_Ch11_01.ipynb, accessed: 2024-11-09
- Wang A, Wu Z, Brooks C, Oney S (2024a) Don't step on my toes: Resolving editing conflicts in real-time collaboration in computational notebooks. In: Proceedings of the 1st ACM/IEEE Workshop on Integrated Development Environments, Association for Computing Machinery, New York, NY, USA, IDE '24, p 47–52, DOI 10.1145/3643796.3648453, URL <https://doi.org/10.1145/3643796.3648453>
- Wang AY, Mittal A, Brooks C, Oney S (2019a) How data scientists use computational notebooks for real-time collaboration. Proceedings of the ACM on Human-Computer Interaction 3, DOI 10.1145/3359141
- Wang F, Lin Y, Yang L, Li H, Gu M, Zhu M, Qu H (2024b) Outlinespark: Igniting ai-powered presentation slides creation from computational notebooks through outlines. In: Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems, Association for Computing Machinery, New York, NY, USA, CHI '24, DOI 10.1145/3613904.3642865, URL <https://doi.org/10.1145/3613904.3642865>
- Wang J, Li L, Zeller A (2019b) Better code, better sharing: on the need of analyzing jupyter notebooks. Proceedings - 2020 ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results, ICSE-NIER 2020 pp 53–56, DOI 10.1145/3377816.3381724, URL <https://arxiv.org/abs/1906.05234v1>

- Wang J, Kuo TY, Li L, Zeller A (2020) Assessing and restoring reproducibility of jupyter notebooks. *Proceedings - 2020 35th IEEE/ACM International Conference on Automated Software Engineering, ASE 2020* 12:138–149, DOI 10.1145/3324884.3416585, URL <https://dl.acm.org/doi/10.1145/3324884.3416585>
- Wang J, Li L, Zeller A (2021) Restoring execution environments of jupyter notebooks. *Proceedings - International Conference on Software Engineering* pp 1622–1633, DOI 10.1109/ICSE43902.2021.00144, URL <https://arxiv.org/abs/2103.02959v1>
- Wang Y, Meijer W, López JAH, Nilsson U, Varró D (2025) Why do machine learning notebooks crash? an empirical study on public python jupyter notebooks. *IEEE Transactions on Software Engineering* 51(7):2181–2196, DOI 10.1109/tse.2025.3574500, URL <http://dx.doi.org/10.1109/TSE.2025.3574500>
- Wang ZJ, Munechika D, Lee S, Chau DH (2024c) Supernova: Design strategies and opportunities for interactive visualization in computational notebooks. In: *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, Association for Computing Machinery, New York, NY, USA, CHI EA '24, DOI 10.1145/3613905.3650848, URL <https://doi.org/10.1145/3613905.3650848>
- Weber T, Mayer S (2024) From computational to conversational notebooks. In: *Proceedings of the 1st CHI Workshop on Human-Notebook Interactions* (Honolulu, Hawaii, USA, 2024-05-11). Online
- Weinman N, Barik T, Drucker SM (2021) Fork it: Supporting stateful alternatives in computational notebooks. *Conference on Human Factors in Computing Systems - Proceedings* DOI 10.1145/3411764.3445527, URL <https://dl.acm.org/doi/10.1145/3411764.3445527>
- Zhu C, Saha RK, Prasad MR, Khurshid S (2021) Restoring the executability of jupyter notebooks by automatic upgrade of deprecated apis. *Proceedings - 2021 36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021* pp 240–252, DOI 10.1109/ASE51524.2021.9678889
- Zhu JS, Zhang Z, Zhao J (2024) Facilitating mixed-methods analysis with computational notebooks. URL <https://arxiv.org/abs/2405.19580>, 2405.19580